BCNF Explanation

Three of the 12 tables from 3NF violated BCNF: `Project_Supplier_Equipment`, `Project_Supplier_Materials` and `Projects`. We decomposed each one, and the other nine already satisfied BCNF.

Functional dependencies checked

We took each of the 12 tables from 3NF, identified its candidate key, and listed the dependencies in it. We also ran a script (`check_bcnf.py`) that tests every one- and two-column determinant against the data. We then judged each result against how the data works in the real world, because the dataset is small and produces accidental matches.

Determinants that were not candidate keys

Yes, we found three real ones:

- `Project_Supplier_Equipment` (key: `ProjectID, SupplierName, EquipmentUsed`): `ProjectID + EquipmentUsed → EquipmentRentalCost`. The rental cost is the same whichever supplier provides the item. For example, a Crane on P001 costs 5000 from both BuildPro Supplies and SteelWorks Inc.
- `Project_Supplier_Materials` (key: `ProjectID, SupplierName, MaterialSupplied`): `ProjectID + MaterialSupplied → MaterialUnitCost`, for the same reason.
- `Projects` (key: `ProjectID`): `SiteCity → SiteState`. A city belongs to one state, but city is not a key of Projects.

How we decomposed

- `Project_Equipment(ProjectID, EquipmentUsed, EquipmentRentalCost)`, PK `(ProjectID, EquipmentUsed)`.
- `Project_Supplier_Equipment(ProjectID, SupplierName, EquipmentUsed)`, PK all three columns. FK `(ProjectID, EquipmentUsed)` references `Project_Equipment`. FK `(ProjectID, SupplierName)` references `Project_Suppliers`.
- `Project_Materials(ProjectID, MaterialSupplied, MaterialUnitCost)`, PK `(ProjectID, MaterialSupplied)`.
- `Project_Supplier_Materials(ProjectID, SupplierName, MaterialSupplied)`, PK all three columns. FK `(ProjectID, MaterialSupplied)` references `Project_Materials`. FK `(ProjectID, SupplierName)` references `Project_Suppliers`.
- `Cities(SiteCity, SiteState)`, PK `SiteCity`. Projects no longer has `SiteState` and keeps `SiteCity` as an FK to `Cities`.

Each split is lossless. Our script joined each pair of new tables back together and got exactly the original rows.

Candidate keys after decomposition

Table	Candidate key
Project_Equipment	(ProjectID, EquipmentUsed)
Project_Supplier_Equipment	(ProjectID, SupplierName, EquipmentUsed)
Project_Materials	(ProjectID, MaterialSupplied)
Project_Supplier_Materials	(ProjectID, SupplierName, MaterialSupplied)
Cities	SiteCity
Projects	ProjectID
Clients	ClientName
Workers	WorkerName
Suppliers	SupplierName
Supervisors	SupervisorName
Project_Suppliers	(ProjectID, SupplierName)
Project_Assignments	(ProjectID, WorkerName)
Assignment_Skills	(ProjectID, WorkerName, WorkerSkill)
Assignment_Certifications	(ProjectID, WorkerName, Certification)
Supplier_Phones	SupplierPhone (each number belongs to one supplier)

Dependencies we rejected as coincidence

The script still reports `ClientName → ProjectType` in Projects and `MaterialUnitCost → MaterialSupplied` in Project_Materials. These appear only because the sample has few rows. A client can have projects of different types, and many materials can share a price, so they are not real rules and we did not decompose on them. The script also reported odd candidate keys, such as `StartDate` in Projects. The real keys are the ones listed above.

Why the design is in BCNF

In every table, each real determinant is a candidate key. The three violations were removed by decomposition, and the remaining tables were checked and already satisfied BCNF.

Note on earlier stages

The equipment and materials dependencies were partial dependencies on the three-column key, so they could also have been caught at 2NF. BCNF is where we found and fixed them.